Universidad Autónoma del Estado de Hidalgo

CENTRO DE CÓMPUTO ACADÉMICO

MANUAL DE INSTALACIÓN

Elaborado por:

ISRAEL GARCIA AYALA Y URIEL IVAN PEREZ TRUJILLO

Introducción

El presente documento tiene como finalidad describir de manera detallada el proceso de instalación, configuración y uso del sistema denominado Chatbot de Álgebra, una aplicación web orientada al apoyo académico en el área de matemáticas.

Este sistema integra tecnologías modernas de desarrollo web junto con inteligencia artificial local, permitiendo la interacción con un asistente capaz de responder y explicar problemas algebraicos en tiempo real.

La guía está diseñada para que cualquier usuario con conocimientos básicos en computación pueda implementar correctamente el sistema sin errores.

Descripción del sistema

El sistema consiste en una aplicación web desarrollada bajo una arquitectura moderna que combina frontend, backend y procesamiento de inteligencia artificial.

Para la capa de desarrollo se utiliza Next.js junto con React, permitiendo una experiencia dinámica para el usuario. La persistencia de datos se maneja mediante SQLite, mientras que Prisma ORM facilita la comunicación con la base de datos.

Para la generación de respuestas inteligentes, el sistema utiliza **Ollama**, el cual ejecuta modelos de lenguaje de forma local, como por ejemplo **Llama 3**.

Arquitectura del sistema

El sistema del Chatbot de Álgebra está basado en una arquitectura cliente-servidor con integración de inteligencia artificial local. Esta arquitectura permite separar la interacción del usuario, la lógica del sistema y el procesamiento de respuestas inteligentes.

El sistema se divide en tres capas principales. La primera corresponde a la capa de presentación, desarrollada con Next.js y React, la cual permite al usuario interactuar mediante una interfaz web. La segunda es la capa de aplicación, encargada de procesar las solicitudes, gestionar la lógica del sistema y comunicarse con los diferentes servicios.

Finalmente, la capa de datos e inteligencia artificial utiliza SQLite para el almacenamiento de información y **Ollama** junto con modelos como **Llama 3** para generar respuestas.

Esta estructura permite una organización clara del sistema y facilita su mantenimiento y escalabilidad.

Flujo del funcionamiento

El sistema opera mediante un flujo de interacción continuo entre el usuario y los diferentes componentes internos.

En primer lugar, el usuario ingresa una pregunta en la interfaz web. Posteriormente, la solicitud es enviada al backend, donde es procesada y dirigida al servicio de inteligencia artificial. A continuación, el modelo de lenguaje genera una respuesta basada en la información recibida. Finalmente, la respuesta es enviada de vuelta al frontend, donde se muestra al usuario y puede almacenarse en la base de datos.

Este flujo permite una interacción dinámica y en tiempo real.

Componentes del sistema

El sistema está compuesto por diversos componentes que trabajan de manera conjunta para garantizar su funcionamiento.

Entre los principales componentes se encuentran la interfaz de usuario, que permite la interacción directa con el sistema; el backend o API, que gestiona la lógica de negocio; el módulo de inteligencia artificial, encargado de procesar las consultas; y la base de datos, que almacena la información generada.

Cada uno de estos elementos cumple una función específica, contribuyendo al correcto desempeño del sistema.

Diseño del sistema

El diseño del sistema se basa en principios de modularidad, lo que permite dividir el sistema en partes independientes que pueden ser desarrolladas y mantenidas de manera separada.

La comunicación entre los diferentes componentes se realiza mediante peticiones HTTP y consultas a la base de datos a través de Prisma, lo que facilita la integración entre frontend, backend y servicios externos.

Este enfoque permite una mayor flexibilidad y facilita la incorporación de nuevas funcionalidades.

Escalabilidad

El sistema ha sido diseñado para permitir futuras mejoras sin afectar su funcionamiento actual.

Entre las posibles extensiones se incluyen la integración de nuevos modelos de inteligencia artificial, la implementación de sistemas de autenticación de usuarios y la migración a bases de datos más robustas en caso de ser necesario.

Requisitos del sistema

Para garantizar el correcto funcionamiento, es necesario contar con los siguientes elementos:

Requisitos de hardware

Se recomienda un equipo con al menos 8 GB de memoria RAM, aunque para un mejor rendimiento se sugieren 16 GB, así como al menos 5 GB de espacio libre en disco o más.

Requisitos de software

Es necesario tener instalado Node.js (versión 18 o superior), npm, Git, un navegador web moderno y el software **Ollama**, el cual es indispensable para la generación de respuestas mediante inteligencia artificial.

1. Instalación del Entorno de Desarrollo

1.1 Instalación de Node.js

Se debe descargar e instalar Node.js desde su sitio oficial. Se recomienda utilizar la versión LTS por su estabilidad.

Una vez instalado, se puede verificar su correcta instalación ejecutando en la terminal:

```
node -v
npm -v
```

1.2 Instalación de Git

Git permite la descarga y gestión del proyecto desde un repositorio.

Para verificar su instalación, se debe ejecutar:

```
git --version
```

2. Descarga del proyecto

El proyecto puede descargarse desde un repositorio mediante el siguiente comando:

```
git clone https://github.com/ivanperezt270-wq/ProyectoChatbotApoyoAcademico.git
```

Posteriormente, se debe acceder al directorio del proyecto.

3. Instalación de dependencias

Dentro del proyecto, se deben instalar todas las dependencias ejecutando:

```
npm install
```

Este proceso descargará automáticamente todas las librerías necesarias.

4. Configuración de la base de datos

El sistema utiliza Prisma como herramienta para la gestión de la base de datos SQLite.

En caso de ser necesario, se puede instalar Prisma mediante:

```
npm install prisma --save-dev
npm install @prisma/client
```

Posteriormente, se configura el archivo .env con la siguiente variable:

```
DATABASE_URL="file:./dev.db"
```

Una vez configurado, se deben ejecutar los siguientes comandos:

```
npx prisma generate
npx prisma db push
```

Con esto se generará el cliente de Prisma y se creará la base de datos junto con sus tablas.

5. Visualización de la base de datos

Para administrar la base de datos de forma visual, se puede utilizar Prisma Studio mediante el siguiente comando:

```
npx prisma studio
```

Esto abrirá una interfaz en el navegador donde se pueden visualizar y modificar los datos almacenados.

6. Instalación del sistema de inteligencia artificial

El sistema requiere obligatoriamente la instalación de **Ollama**, el cual permite ejecutar modelos de lenguaje de manera local.

Una vez instalado, se puede verificar su funcionamiento ejecutando:

```
ollama –version
```

7. Descarga del modelo de lenguaje

Se debe descargar el modelo necesario, como por ejemplo **Llama 3**, mediante el siguiente comando:

```
ollama pull llama3
```

8. Ejecución del servicio de inteligencia artificial

Antes de iniciar el sistema, se debe ejecutar el siguiente comando:

```
ollama serve
```

Esto permitirá que el backend pueda comunicarse con el modelo de lenguaje.

Para validar su funcionamiento, se puede ejecutar:

```
curl http://localhost:11434
```

9. Configuración final

El archivo .env debe contener la siguiente configuración:

```
DATABASE_URL="file:./dev.db"
OLLAMA_URL="http://localhost:11434"
MODEL="llama3"
```

10. Ejecución del sistema

Para iniciar la aplicación, se deben seguir los siguientes pasos:

Primero, ejecutar el servicio de inteligencia artificial:

```
ollama serve
```

Después, iniciar el proyecto:

```
npm run dev
```

Finalmente, abrir un navegador web y acceder a:

http://localhost:3000

Uso del sistema

El usuario puede interactuar con el chatbot ingresando preguntas relacionadas con álgebra. El sistema procesará la solicitud, generará una respuesta mediante inteligencia artificial y la mostrará en la interfaz.

Pruebas del sistema

Una vez finalizada la instalación, es importante verificar que el sistema funciona correctamente mediante las siguientes pruebas.

1 Prueba de ejecución del servidor

Se debe ejecutar el proyecto con el comando:

```
npm run dev
```

Si el sistema inicia sin errores y muestra la URL local, significa que el servidor está funcionando correctamente.

2 Prueba de acceso a la aplicación

Abrir un navegador web e ingresar a la dirección:

http://localhost:3000

Si la interfaz del chatbot se visualiza correctamente, la aplicación está operativa.

3 Prueba de conexión con la base de datos

Ejecutar el siguiente comando:

```
npx prisma studio
```

Si se abre la interfaz de Prisma Studio en el navegador y se visualizan las tablas, la conexión con la base de datos es correcta.

4 Prueba del servicio de inteligencia artificial

Verificar que el servicio de **Ollama** esté activo ejecutando:

ollama serve

Posteriormente, realizar una prueba directa con el modelo:

ollama run llama3

Si el modelo responde correctamente, el sistema de inteligencia artificial está funcionando.

5 Prueba de interacción completa

Ingresar una pregunta en el chatbot, por ejemplo:

“Resuelve la ecuación x + 5 = 10”

Si el sistema genera una respuesta adecuada, se confirma que la integración completa (frontend, backend, base de datos e inteligencia artificial) funciona correctamente.

21. Checklist de validación final

Antes de dar por concluida la instalación, se recomienda verificar los siguientes puntos:

- Node.js instalado y funcionando
- Git instalado correctamente
- Proyecto descargado desde el repositorio
- Dependencias instaladas sin errores
- Archivo .env configurado correctamente
- Base de datos creada mediante Prisma
- Prisma Studio funcional
- Servicio de **Ollama** en ejecución
- Modelo de lenguaje descargado (llama3)
- Aplicación ejecutándose en el navegador

- Respuestas del chatbot funcionando correctamente

“**Problemas comunes y soluciones”**

Durante el proceso de instalación y ejecución del sistema, pueden presentarse algunos errores. A continuación, se describen los más comunes junto con sus respectivas soluciones.

1. **Error en la instalación de dependencias**

En algunos casos, al ejecutar la instalación del proyecto pueden faltar paquetes o generarse errores.

Solución:
Se recomienda ejecutar nuevamente la instalación de dependencias. En caso de persistir el problema, eliminar la carpeta de dependencias y reinstalar.

Comandos:

```
npm install
```

o bien:

Eliminar la carpeta node_modules y volver a ejecutar la instalación.

2. **Error en la configuración de Prisma**

Puede ocurrir que Prisma no genere correctamente el cliente o que la base de datos no exista.

Solución:
Se deben ejecutar nuevamente los comandos de generación y sincronización de la base de datos.

Comandos:

```
npx prisma generate
npx prisma db push
```

3. **Error: base de datos no encontrada**

Este error ocurre cuando el archivo de base de datos no ha sido creado.

Solución:
Verificar que la variable DATABASE_URL esté correctamente configurada y ejecutar nuevamente la sincronización con Prisma.

4. **Error con el archivo .env**

Una configuración incorrecta en las variables de entorno puede impedir la conexión con la base de datos o con el servicio de inteligencia artificial.

Solución:
Verificar que el archivo .env exista en la raíz del proyecto y que las variables estén correctamente escritas.

5. **Error: servicio de IA no disponible**

Si el sistema no genera respuestas, es probable que el servicio de inteligencia artificial no esté en ejecución.

Solución:
Iniciar el servicio de **Ollama** antes de ejecutar el proyecto.

Comando:

```
ollama serve
```

6. Error: modelo de lenguaje no encontrado

Este error ocurre cuando el modelo de inteligencia artificial no ha sido descargado previamente.

Solución:

Descargar el modelo correspondiente.

Comando:

```
ollama pull llama3
```

7. Error de conexión con el servicio de IA

Puede presentarse un error cuando el sistema no logra conectarse al servicio de inteligencia artificial.

Solución:

Verificar que el servicio esté activo en la dirección correcta (http://localhost:11434) y que no existan bloqueos de red o puertos.

8. Error: puerto en uso

Si el sistema no inicia correctamente, puede deberse a que el puerto por defecto ya está ocupado.

Solución:

Ejecutar el proyecto en un puerto diferente.

Comando:

```
npm run dev -- -p 3001
```

9. Error de rendimiento lento

En algunos equipos, el sistema puede responder lentamente debido al uso del modelo de inteligencia artificial.

Solución:

Cerrar aplicaciones en segundo plano o utilizar un equipo con mayor capacidad de memoria RAM.

La mayoría de los errores durante la instalación del sistema están relacionados con dependencias, configuración del entorno o ejecución del servicio de inteligencia artificial. Siguiendo las soluciones descritas, es posible resolverlos de manera rápida y garantizar el correcto funcionamiento del sistema.

Conclusión General

El desarrollo e implementación del Chatbot de Álgebra representa una integración efectiva de tecnologías modernas de desarrollo web e inteligencia artificial, orientadas a la creación de una herramienta educativa funcional y accesible.

A lo largo del presente manual se detalló el proceso completo de instalación, configuración y ejecución del sistema, incluyendo la preparación del entorno, la gestión de la base de datos mediante Prisma y la integración del servicio de inteligencia artificial a través de **Ollama** con modelos como **Llama 3**. Esto permite que el sistema opere de manera autónoma, sin depender de servicios externos, garantizando mayor control y privacidad.

Asimismo, se establecieron procedimientos para la validación del sistema, así como la identificación y solución de errores comunes, asegurando que el proceso de instalación pueda realizarse de manera clara y sin inconvenientes.

En conclusión, el sistema presenta una arquitectura sólida, modular y escalable, lo que facilita su mantenimiento y futuras mejoras. Su correcta implementación permite ofrecer una experiencia interactiva orientada al aprendizaje, posicionándolo como una solución tecnológica viable dentro del ámbito Educativo